

Výkonnost, RVP

INP - cvičení 12

Zdeněk Vašíček, 2021
vasicek@fit.vutbr.cz



Relativní výkonnost

- Zvolíme **referenční počítač**
- Zvolíme **sadu testovacích úloh** (tzv. benchmarky)
- Spočítáme **relativní doby výpočtu úloh** na daném počítači (tj. vzhledem k referenčnímu počítači)
- Z relativních dob můžeme určit a použít ke srovnání:
 - aritmetický průměr
 - vážený průměr
 - geometrický průměr
- Obvykle je používán **geometrický průměr** (viz sady testovacích úloh SPEC, Whetstone, Dhrystone, ...)

Sada testovacích úloh SPEC89

Benchmarks	Type	Application Description
001.gcc	INT ¹	GNU C compiler
008.espresso	INT	PLA optimizing tool
013.spice2g6	FP ²	Circuit simulation and analysis
015.doduc	FP	Monte Carlo simulation
020.nasa7	FP	Seven floating-point kernels
022.li	INT	LISP interpreter
023.eqntott	INT	Conversions of equations to truth table
030.matrix300	FP	Matrix solutions
042.fpppp	FP	Quantum chemistry application
047.tomcatv	FP	Mesh generation application

INT = Integer intensive benchmark.

FP = Double-precision floating-point benchmark.

$$spec_{int\ 89} = \sqrt[4]{\prod_{i=1}^4 t_{r_i}} \qquad spec_{fp\ 89} = \sqrt[6]{\prod_{i=1}^6 t_{r_i}}$$

Srovnání výkonnosti dvou systémů s využitím SPEC89

SPEC_{int89} (4 programy), SPEC_{fp89} (6 programů)

SPEC INT SPEC FP	Počítač Úloha	VAX11/780 (s)	DEC3100 (s)	SPARC (s)	DEC3100 (relativní)	SPARC (relativní)
	GCC	1482	145	138,9	10,22	10,67
	Espresso	2266	194	254	11,68	8,92
	Li	6206	480	689,5	12,93	9,08
	Eqntott	1101	99	113,5	11,12	9,7
	Spice	23951	2500	2875,5	9,58	8,33
	Doduc	1863	208	374,1	8,96	4,98
	NASA7	20093	1646	2308,2	12,21	8,71
	Matrix300	4525	749	409,3	6,04	11,06
	Fpppp	3038	292	387,2	10,4	7,85
	Tomcatv	2649	260	469,8	10,19	5,64
		Referenční počítač		SPEC-FX	11,45	9,57
			SPEC-FP	9,36	7,49	

DEC3100 je výkonnější než SPARC

Sada testovacích úloh SPEC CPU2006

<http://www.spec.org/cpu2006/results/>

SPECINT 2006 (12 benchmarků)

Benchmark	Language	Application Area
400.perlbench	C	Programming Language
401.bzip2	C	Compression
403.gcc	C	C Compiler
429.mcf	C	Combinatorial Optimization
445.gobmk	C	Artificial Intelligence: Go
456.hmmer	C	Search Gene Sequence
458.sjeng	C	Artificial Intelligence: chess
462.libquantum	C	Physics / Quantum Computing
464.h264ref	C	Video Compression
471.omnetpp	C++	Discrete Event Simulation
473.astar	C++	Path-finding Algorithms
483.xalancbmk	C++	XML Processing

SPECFP 2006 (17 benchmarků)

Benchmark	Language	Application Area
410.bwaves	Fortran	Fluid Dynamics
416.gamess	Fortran	Quantum Chemistry.
433.milc	C	Physics / Quantum Chromodynamics
434.zeusmp	Fortran	Physics / CFD
435.gromacs	C, Fortran	Biochemistry / Molecular Dynamics
436.cactusADM	C, Fortran	Physics / General Relativity
437.leslie3d	Fortran	Fluid Dynamics
444.namd	C++	Biology / Molecular Dynamics
447.dealII	C++	Finite Element Analysis
450.soplex	C++	Linear Programming, Optimization
453.povray	C++	Image Ray-tracing
454.calculix	C, Fortran	Structural Mechanics
459.GemsFDTD	Fortran	Computational Electromagnetics
465.Tonto	Fortran	Quantum Chemistry
470.lbm	C	Fluid Dynamics
481.wrf	C, Fortran	Weather
482.sphinx3	C	Speech recognition

$$\text{SPEC}_{\text{int 2006}} = \sqrt[12]{\prod_{i=1}^{12} t_{\text{rel}_i}} \quad t_{\text{rel}_i} = \frac{t_{\text{ref}_i}}{t_{\text{act}_i}}$$

$$\text{SPEC}_{\text{fp 2006}} = \sqrt[17]{\prod_{i=1}^{17} t_{\text{rel}_i}} \quad t_{\text{rel}_i} = \frac{t_{\text{ref}_i}}{t_{\text{act}_i}}$$

Sada testovacích úloh SPEC CPU2017

<http://www.spec.org/cpu2017/results/>

SPEC CPU2017 - 43 benchmarků rozdělených do 4 skupin

- SPECSpeed – zaměřeno na dobu zpracování
- SPECrate – zaměřeno na propustnost

SPECrate 2017 Integer	SPECSpeed 2017 Integer	Language	Application Area
500.perlbench_r	600.perlbench_s	C	Perl interpreter
502.gcc_r	602.gcc_s	C	GNU C compiler
505.mcf_r	605.mcf_s	C	Route planning
520.omnetpp_r	620.omnetpp_s	C++	Discrete Event simulation - computer network
523.xalancbmk_r	623.xalancbmk_s	C++	XML to HTML conversion via XSLT
525.x264_r	625.x264_s	C	Video compression
531.deepsjeng_r	631.deepsjeng_s	C++	Artificial Intelligence: alpha-beta tree search (Chess)
541.leela_r	641.leela_s	C++	Artificial Intelligence: Monte Carlo tree search (Go)
548.exchange2_r	648.exchange2_s	Fortran	Artificial Intelligence: recursive solution generator (Sudoku)
557.xz_r	657.xz_s	C	General data compression

SPECrate 2017 Floating Point	SPECSpeed 2017 Floating Point	Language	Application Area
503.bwaves_r	603.bwaves_s	Fortran	Explosion modeling
507.cactuBSSN_r	607.cactuBSSN_s	C++, C, Fortran	Physics: relativity
508.namd_r		C++	Molecular dynamics
510.parest_r		C++	Biomedical imaging: optical tomography with finite elements
511.povray_r		C++, C	Ray tracing
519.lbm_r	619.lbm_s	C	Fluid dynamics
521.wrf_r	621.wrf_s	Fortran, C	Weather forecasting
526.blender_r		C++, C	3D rendering and animation
527.cam4_r	627.cam4_s	Fortran, C	Atmosphere modeling
	628.pop2_s	Fortran, C	Wide-scale ocean modeling (climate level)
538.imagick_r	638.imagick_s	C	Image manipulation
544.nab_r	644.nab_s	C	Molecular dynamics
549.fotonik3d_r	649.fotonik3d_s	Fortran	Computational Electromagnetics
554.roms_r	654.roms_s	Fortran	Regional ocean modeling

Měření výkonnosti

$$t_e = N_I \cdot \text{CPI} \cdot T_{\text{clk}} = N_I \cdot \text{CPI} / f_{\text{clk}}$$

- t_e – doba výpočtu (vykonávání programu)
- N_I – počet instrukcí
- T_{clk} – perioda hodinového signálu ($T_{\text{clk}} = 1 / f_{\text{clk}}$)
- CPI – počet hodinových cyklů na instrukci (**C**ycles **P**er **I**nstruction)

$$P_{\text{MIPS}} = N_I / (t_e \cdot 10^6) = f_{\text{clk}} / (\text{CPI} \cdot 10^6) \text{ (instrukcí za sekundu v milionech)}$$

$$P_{\text{MFLOPS}} = N_{\text{FPI}} / (t_e \cdot 10^6) \text{ (FP instrukcí za sekundu v milionech)}$$

$$\text{Relativní } P_{\text{MIPS}} = t_{e \text{ ref. poč.}} \cdot P_{\text{MIPS ref. poč.}} / t_{e \text{ měř. Poč.}}$$

Příklad 1

Na počítačích A a B jsme spustili program. Naměřili jsme $t_A=10s$, $t_B=15s$.

O kolik % je A výkonnější než B?

$$P_A/P_B = t_B/t_A = 15/10 = 1,5$$

Příklad CPI vs doba výpočtu

Optimalizující kompilátor odstraní 50% instrukcí ALU, ostatní instrukce ponechá. $T_{\text{clk}}=2\text{ns}$.

1. Vypočtete $\text{CPI}_{\text{pův}}$, CPI_{opt} , $P_{\text{MIPS}_{\text{pův}}}$, $P_{\text{MIPS}_{\text{opt}}}$.

$$\text{CPI}_{\text{pův}} = 0.43 \cdot 1 + 0.21 \cdot 2 + 0.12 \cdot 2 + 0.24 \cdot 2 \cong 1.57$$

$$\text{CPI}_{\text{opt}} = (0.215 \cdot 1 + 0.21 \cdot 2 + 0.12 \cdot 2 + 0.24 \cdot 2) / 0.785 \cong 1.73$$

$$P_{\text{MIPS}_{\text{pův}}} = f_{\text{clk}} / (\text{CPI}_{\text{pův}} \cdot 10^6) = 10^3 / (\text{CPI}_{\text{pův}} \cdot T_{\text{clk}}) \cong 318.4$$

$$P_{\text{MIPS}_{\text{opt}}} = f_{\text{clk}} / (\text{CPI}_{\text{opt}} \cdot 10^6) = 10^3 / (\text{CPI}_{\text{opt}} \cdot T_{\text{clk}}) \cong 289.0$$

$$t_{\text{e,pův}} = N_{\text{I}} \cdot \text{CPI}_{\text{pův}} \cdot T_{\text{clk}} \cong 3.14 \cdot 10^{-9} \cdot N_{\text{I}}$$

$$t_{\text{e,opt}} = 0.785 \cdot N_{\text{I}} \cdot \text{CPI}_{\text{opt}} \cdot T_{\text{clk}} \cong 2.7 \cdot 10^{-9} \cdot N_{\text{I}}$$

Instr.	Četnost	CPI	Opt.
ALU	43 %	1	21.5%
Load	21 %	2	21%
Store	12 %	2	12%
Jmp	24 %	2	24%
	100 %		78.5 %

2. Souhlasí poměr P_{MIPS} a t_{e} ? Co je tu zvláštní?

Výhodnost koupě nového CPU

Procesor je využit z 50%, zbytek času čeká na I/O operace. Cena procesoru tvoří 1/3 ceny počítače. Je výhodné koupit 5x rychlejší CPU za 5x vyšší cenu?

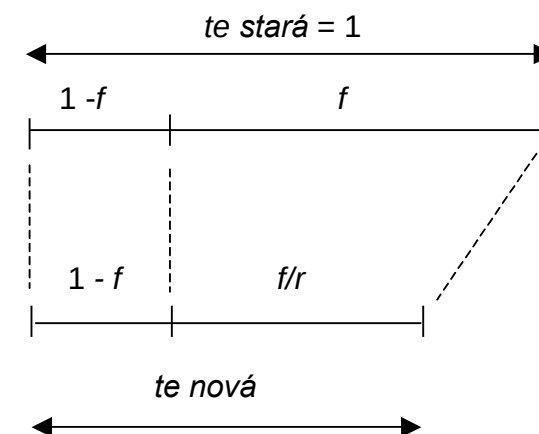
$$f = 0.5, \quad r = 5$$

zrychlení:

$$V_r = 1 / ((1 - f) + f/r) = 1 / ((1 - 0.5) + 0.5/5) = 1.66$$

násobek ceny počítače s novým CPU:

$$k = 2/3 + 5 \cdot 1/3 = 2.33$$



Amdahlův zákon: celkové
zrychlení je rovno

$$V_r = 1 / ((1 - f) + f/r)$$

Příkon CPU

Mějme procesor Athlon XP 1700+, který pracuje na frekvenci 1.466 GHz a napětí 1.75V. Jak se změní výkonnost a spotřeba, pokud procesor podtaktujeme na 1GHz (1.4 GHz, 1.266 GHz) a snížíme napájecí napětí na 1.15V (1.6, 1.45V).

Předpokládejme, že spotřebu na 1.75V lze aproximovat vztahem

$$P = 20 + 0.03 * f \text{ v MHz}$$

Pro příkon platí $P \sim C U^2 f$

$$P_{\text{puv}} \sim C 1.75^2 1466$$

$$P_{\text{puv}} = 20 + 0.03 * 1466 = 64 \text{ W}$$

$$P_{\text{novy}} \sim C 1.15^2 1000$$

$$P_{\text{novy}} / P_{\text{puv}} = 1.15^2 1000 / 1.75^2 1466 = 0.29$$

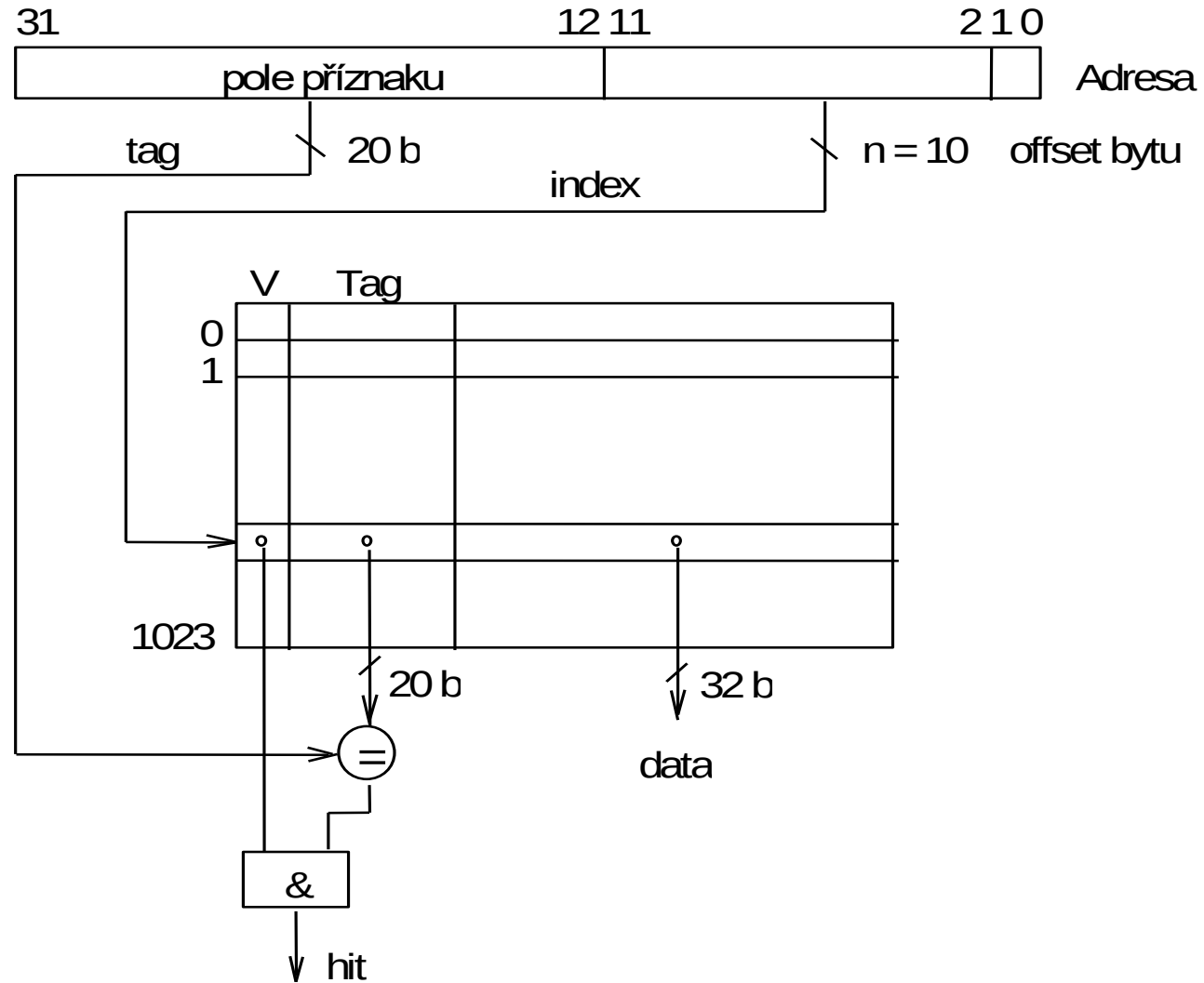
$$\Rightarrow P_{\text{novy}} = 64 * 0.29 = 18.56 \text{ W}$$

$$f_{\text{novy}} / f_{\text{puv}} = 1000 / 1466 = 0.68$$

Spotřeba klesla o 71% zatímco výkon CPU pouze o 32%.

CACHE (RVP)

přehled koncepce s 32-bitovou adresou a přímým mapováním



Příklad 1

Zadání: Kolik kb paměti bude potřeba pro realizaci jednocestné RVP obsahující 1024 bloků? Uvažujte velikost bloku 64 bitů a 32-bitové adresování.

Řešení

Počet položek ... 1024

Počet bitů na jednu položku

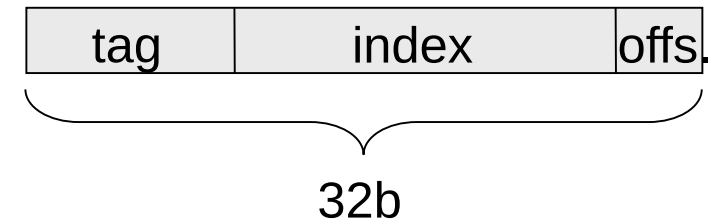
Blok dat ... 64 bitů

Uložení tagu ... $32 - \log_2(1024) - \log_2(64/8) = 19$ bitů

Valid bit ... 1 bit

Celkem ... 84 bitů

Velikost RVP ... $1024 * 84 = 84$ kb



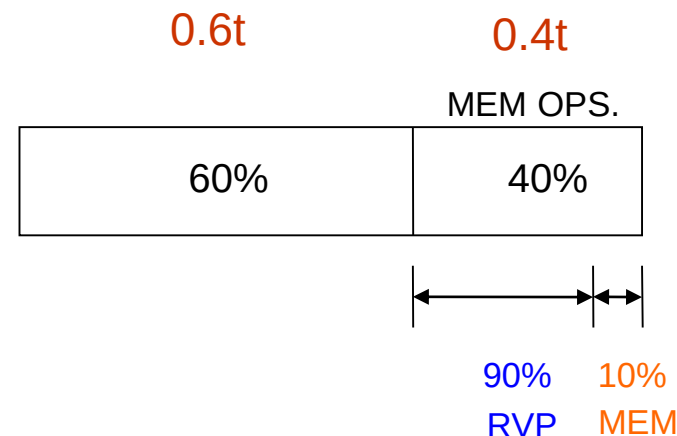
Příklad 2

Zadání: Paměťové operace tvoří 40 % času z celkové doby běhu programu. O kolik procent se zkrátí doba běhu programu při použití RVP, když $p_{\text{hit}} = 0,9$ a doba přístupu do RVP je rovna 1/10 doby přístupu do paměti?

$$t_{\text{puvodni}} = t$$

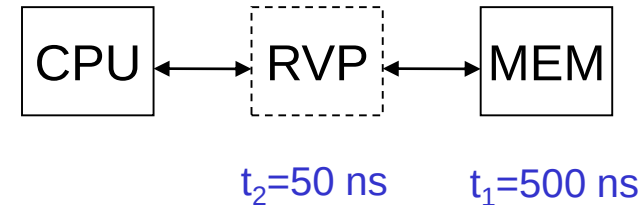
$$t_{\text{RVP}} = 0,6 \cdot t + 0,4 \cdot (0,1 \cdot t + 0,9 \cdot 0,1 \cdot t) = 0,676 \cdot t$$

$$t_{\text{puvodni}} / t_{\text{RVP}} = t / 0,676 \cdot t = 1,479$$



Příklad 3

Zadání: Mezi procesor a paměť s dobou přístupu 500 ns byla vložena RVP s dobou přístupu 50 ns. Jaká musí být pravděpodobnost zásahu p_{hit} do RVP, aby bylo dosaženo alespoň dvojnásobné urychlení paměťových operací?



$$t_{\text{puvodni}} = t_1 \cdot n = 500 \cdot n$$

$$t_{\text{RVP}} = (t_2 \cdot p_{hit} + (t_2 + t_1) \cdot (1 - p_{hit})) \cdot n = (-t_1 \cdot p_{hit} + t_1 + t_2) \cdot n = (550 - 500 \cdot p_{hit}) \cdot n$$

$$t_{\text{puvodni}} / t_{\text{RVP}} = 500 \cdot n / (550 - 500 \cdot p_{hit}) \cdot n = 500 / (550 - 500 \cdot p_{hit}) = 2$$

$$p_{hit} = (1100 - 500) / 1000 = 3/5 = 0,6$$